CS4001: Cloud and Fog Computing for IoT

CIA - 2

Priyajit Biswal

Registration No: 23011102068

GitHub Repository: [priyajitbiswal/mobility-aware-fog-simulation]

1. Introduction

Fog computing aims to provide cloud-like computation and storage capabilities closer to mobile end-users by deploying server nodes at the network edge (e.g., Roadside Units (RSUs) and 5G cellular base stations). Within smart/connected transportation and vehicular environments, connected vehicles and smartphones are continuously generating a spectrum of tasks, ranging from timely-critical driving safety tasks (e.g., collision alerts, driving perception for autonomous driving) to general navigation and traffic-related tasks.

Due to the nature of vehicles moving at speeds of 40 km/h to 100 km/h, and roadside fogs having coverage radius of 150 to 300 meters, task offloading can lead to disconnections (due to out-of-coverage) during execution, requiring the fog node to re-establish connection and reroute previously offloaded tasks.

Moreover, connected vehicles and smartphones are equipped with On-Board Units (OBUs) and mobile devices with limited battery and energy budgets, and task migration requires additional wireless and backhaul energy consumption.

Therefore, an efficient fog computing framework must be both mobility- and energy-aware. This report conducts a survey of 10 research papers, identifies the challenges, and proposes a balanced three-tier architecture (named Mobility-Aware Efficient Fog Computing (MAEFC)) that enables reliable task completion while preserving energy efficiency.

2. Literature Survey (10 Research Papers)

To evaluate how existing literature addresses mobility prediction, task migration, and energy-efficient resource management, 10 representative research papers were reviewed:

1. Tianyi Zhu, Ting Shi, Jianxun Li, Zhipeng Cai, and Xiangke Zhou (2019) - 'Fog Offloading with Latency and Quality Optimization'

The authors conducted a study of task offloading in a network consisting of both stationary infrastructure and moving vehicles. They formulated the problem of choosing where to offload tasks as a Linear Programming + Binary Particle Swarm Optimization problem, seeking to minimize

latency and video quality degradation with respect to real-world GPS traces of San Francisco taxis. The resulting scheme reduced average latency by 27% and video quality loss by 56% over randomly chosen assignments, though it does not account for coordination with cloud tiers and makes no attempt to predictively model future vehicle positions.

2. Wei Shi, Jiannong Cao, Quan Zhang, Yang Liu, and Lu Xu (2018) - 'Mobility-Aware Genetic Algorithm for Multi-Site Task Offloading (MAGA)'
The researchers designed the Genetic Algorithm (GA) to offload tasks in distributed edge fog environments considering the mobility of vehicles and deadlines. The fitness function was defined using linear speed assumptions considering the delay, energy consumption of devices, and task completion rate. Although MAGA improved the balance between the delay and the success rate of the algorithm compared to greedy approaches, such algorithms require extensive iterative calculations, making them impossible for real-time use, and the cloud was not integrated.

3. Diganta Kakati, Samarjeet Borah, and Sanchita Barman (2022) - 'Mobility-Aware Proactive Task Offloading: Eliminating Migration Overhead'
The authors proposed an offloading strategy that forbids task migration altogether. By proactively partitioning and scheduling tasks so that they complete before the vehicle exits the cell, they achieved a 44% reduction in latency and 68% reduction in edge network utilization by eliminating migration overhead. However, forbidding migration leads to tasks being dropped in the case that vehicles accelerate prematurely or tasks complete later than expected and requires over-provisioning of task distribution.

4. Xiaojie Wang, Zhaolong Ning, and Lei Wang (2019) - 'Offloading in Internet of Vehicles: A Fog-Enabled Real-Time Traffic Management System'
In their paper, the authors studied a two-layer fog system that predicts 'sojourn time' (a vehicle's dwell time within a fog cell). A Gini-coefficient load balancing heuristic was used in combination with a genetic algorithm to avoid edge node overloads and minimize handovers. The scheme reduced energy and delay compared to nearest-node offloading but lacks cloud backup, and the probabilistic dwell-time estimates break down in the face of the highly variable urban traffic.

5. Yan Zhang, Jun Liu, and Mianxiong Dong (2024) - 'Extended Kalman Filter-Based Location Prediction for Dynamic Vehicular Fog Offloading'
The researchers used an Extended Kalman Filter (EKF) on kinematic GPS and IMU sensor information to predict positions of vehicles and roadside units for offloading purposes. The vehicle chooses which cooperative vehicle or roadside unit to offload tasks to based on predictions of when it will be near them. This allowed much higher completion rates than before, but the paper notes that constant EKF matrix updates are a computational burden to the vehicle, and battery consumption was not taken into account.

6. Jaewon Lee and Jong-Kook Lee (2020) - 'Resource Allocation in Vehicular Fog Computing Using Reinforcement Learning with Heuristic Information'
The authors utilized Q-learning type of reinforcement learning in combination with domain heuristics (such as queue lengths and node distance) to address edge resource allocation problem. Heuristic hints helped to reduce action-space exploration time and increase utilization of the edge servers. The vehicle mobility was considered only through implicit observation of the environment state, Q-tables grow exponentially with vehicle density and battery conservation of devices was not taken into account.

7. Zhe Chen, Xiaofei Wang, and Bo Rong (2023) - 'Multi-Hop Task Offloading in MEC-Enabled Internet of Vehicles'
The authors propose a multi-hop task offloading framework for vehicular IoT environments where vehicles not in direct RSU range offload tasks to intermediate relay vehicles. This provides extended coverage without requiring additional roadside infrastructure, but introduces cumulative queuing delays and packet jitter, and drains the battery power of the forwarding relay vehicles.

8. Reena Yadav, Weishan Zhang, and He Chen (2020) - 'Joint Task Offloading and Resource Allocation for Energy-Latency Trade-Off in Vehicular Fog'
The researchers formulated a convex relaxation and Lagrangian duality joint optimization problem that considers devices' battery energy and task latency. They proved that the Pareto-optimal time-energy jointly dynamic CPU clock frequencies and offloading decisions are possible and better than sequential optimization. However, the paper does not consider migrating tasks from one device to another and using the cloud computing platform.

9. Lin Tang, Xiang He, and Chen Yang (2020) - 'Vehicles as Fog Nodes: Latency-Optimal Task Assignment in Vehicular Fog Computing with 5G'
The authors proposed to exploit moving and parked vehicles equipped with idle computing power for acting as opportunistic fog servers through high-Speed 5 G connections. Results showed that vehicular compute pools can lower roadside latency and infrastructure costs. However, vehicle availability is opportunistic, vehicle battery degradation was not evaluated, and security/trust concerns of offloading to 3rd party vehicles were not resolved.

10. Mohammad Mahdizadeh, Mohammad Hossein Rezvani, and Team (2025) - 'Multi-Objective Fog Offloading and Resource Allocation Using Evolutionary Game Theory (MOFCO)'
The authors of this paper have developed a multi-objective fog computing offloading model using an evolutionary game theoretic approach. They have ranked potential fog nodes according to a 'Fog Node Evaluation Factor' that takes into account the current load, predicted workload, and vehicle

trajectory. The MOFCO algorithm penalized unnecessary task migration, outperforming genetic algorithms by 19-25% in costs while achieving up to 43% lower costs in heavy traffic conditions, successfully balancing battery conservation with latency.

Surveyed Works Comparison

Authors & Year	Mobility Approach	Key Technique Used	Primary Metrics	Main Limitation
Tianyi Zhu, Ting Shi, et al. (2019)	Taxi GPS traces	Linear Prog. + PSO	Latency, Video Quality	No cloud tier; no prediction
Wei Shi, Jiannong Cao, et al. (2018)	Linear speed model	Genetic Algorithm	Delay, Energy, Success	Slow convergence; no cloud
Diganta Kakati, Samarjeet Borah, et al. (2022)	Mobility-agnostic	Proactive distribution	Latency, Network usage	Bans migration (drops tasks)
Xiaojie Wang, Zhaolong Ning, et al. (2019)	Sojourn dwell time	Gini heuristic + GA	Delay, Energy	Reactive handovers; no cloud tier
Yan Zhang, Jun Liu, et al. (2024)	Extended Kalman Filter	Dynamic cooperative target	Delay, Success rate	GPS overhead; ignores battery energy
Jaewon Lee & Jong-Kook Lee (2020)	Implicit RL state	Heuristic Q-Learning	Delay, Server utilization	Q-table scaling; ignores energy
Zhe Chen, Xiaofei Wang, et al. (2023)	Vehicular multi-hop	Relay path routing	Response time, Energy	Multi-hop delay; drains relay battery
Reena Yadav, Weishan Zhang, et al. (2020)	Simple vehicular	Joint convex optimization	Energy, Latency	No migration; no cloud backup
Lin Tang, Xiang He, et al. (2020)	Vehicle trajectories	5G opportunistic assignment	Latency	Unreliable nodes; no energy model

Authors & Year	Mobility Approach	Key Technique Used	Primary Metrics	Main Limitation
Mohammad Mahdizadeh et al. (2025)	Trajectory + Load	Evolutionary Game Theory	Cost (Delay + Energy + Migr.)	Cloud used only as emergency overflow

3. Research Gap

A comparative review of the 10 research papers reveals three major gaps in the existing literature:

1. Lack of three-tier synergy: Nine out of the ten surveyed papers operate strictly on a two-tier model (Device + Fog), and ignore the remote cloud. Without cloud integration, roadside fog nodes quickly become overloaded during peak traffic, and there is no centralized backup when edge nodes fail, or vehicles travel through rural highway gaps.

2. Migration strategy extremes: Current literature exhibits an all-or-nothing approach to migration. Kakati et al. completely bans migration, which drops tasks if vehicles accelerate. In contrast, Wang et al. trigger migration reactively whenever a vehicle crosses a boundary, which wastes network bandwidth and energy. A selective mechanism that migrates only when the benefit justifies the cost is missing.

3. Oversimplified mobility and incomplete energy accounting: Many existing works rely on straight-road assumptions or static Poisson dwell time. Furthermore, energy models routinely ignore the electrical power consumed by inter-RSU fiber transceivers and memory serialization during task migration.

4. Challenges of mobility support

Providing computing services to mobile vehicles introduces five major challenges:

1. Short contact window & orphaned tasks: Because vehicles move at 40 to 100 km/h, while roadside units cover 150 to 300 meters, communication contact lasts only 10 to 30 seconds. If a task does not finish before the vehicle leaves, the connection drops and the task become orphaned.

2. Latency spikes from triangle routing: If an offloaded task finishes after the vehicle has departed, the result must be routed over multi-hop backhauls to catch up with the vehicle's new base station. This triangle routing raises latency from <15 ms to >80 ms, violating safety deadlines.

3. High migration overhead: Moving an active computation container from one RSU to another, requires transferring memory state, execution stacks, and dirty pages. This consumes backhaul bandwidth and electrical power. If done carelessly, migration consumes more energy than offloading saves.

4. Ping-pong handovers: Where adjacent fog nodes overlap, small signal fluctuations cause vehicles to alternate rapidly between nodes, causing migration thrashing, where tasks are repeatedly moved without making compute progress.

5. Battery limitations and edge load imbalance: Vehicles and smartphones operate under battery limitations while roadside nodes have fixed power supplies. Concentrated traffic at intersections can overwhelm a single fog node while adjacent nodes remain idle.

5. Proposed Architecture: Mobility-Aware, Energy-Efficient Fog Computing (MAEFC)

To resolve these challenges, we design the MAEFC framework, which utilizes a three-tier architecture (Device -> Fog -> Cloud) plus simple dwell-time prediction and selective migration.

Three-Tier Architecture

1. Device Tier (Vehicles / Mobile Users): Vehicles with low-power processors, GPS sensors, and 5G/DSRC radios; responsible for running urgent safety tasks, local battery reserve monitoring, and reporting velocity and heading to roadside fog nodes.

2. Fog Tier (Roadside Units): Roadside Units (RSUs) spaced 300-500 meters apart, connected via 10 Gbps fiber links; equipped with multi-core processors and localized storage; responsible for estimating vehicle dwell times, executing offloaded real-time tasks, and coordinating peer-to-peer task migration.

3. Cloud Tier (Remote Cloud): Centralized hyperscale cloud data centers, connected via WAN; handles heavy, delay-tolerant workloads (long-term route planning, ML training), stores periodic task backups, and takes over overflow tasks when fog nodes are saturated.

Key Functional Components

1. Mobility Predictor: The serving fog node estimates how much time remains before the vehicle exits its coverage zone, using the simple relation: Dwell Time = Remaining Distance to Boundary / Current Speed. This determines whether a task can complete locally.

2. Task Classifier: Tasks are grouped into three clear classes: (a) High-Priority Safety (collision warnings, deadline < 15 ms, executed locally on the vehicle); (b) Real-Time Control (traffic sign

recognition, deadline < 100 ms, offloaded to fog); (c) Background Analytics (map downloads, deadline > 1 s, offloaded to cloud).

3. Offloading Decision Engine: For offloadable tasks, the decision engine evaluates the cost: Cost = w1 (Delay / Max_Delay) + w2 (Energy / Max_Energy) + w3 Migration_Risk. The node or tier with the lowest cost is selected.

4. Task Migration Manager: Instead of always migrating or never migrating, MAEFC applies Selective Migration. If the task finishes before exit, no migration occurs. If disconnection is imminent and next node is available, Proactive Live Migration pre-copies memory. If path is uncertain, it uses Reactive Checkpointing. During coverage gaps, it falls back to Cloud Checkpointing.

5. Fog Node Selector: When offloading to the fog tier, the system selects the node with good signal quality, spare CPU capacity, and expected dwell time long enough to complete the task.

6. Energy Management Module: Fog nodes use Dynamic Voltage and Frequency Scaling (DVFS). Tasks with extra slack time are executed at lower CPU clock speeds to save energy, while urgent tasks run at full frequency.

Operational Workflow

The end-to-end task handling workflow is composed of the following steps:

1. Kinematic Beacon: The vehicle connects to RSU F1 and shares its current speed, heading, and GPS location.

2. Task Arrival & Priority Check: When a task is generated, if it is a safety-critical task (<15 ms), it is executed locally on the vehicle OBU.

3. Dwell Time Estimation: If the task is offloadable, F1 computes vehicle dwell time. If dwell time exceeds execution time, F1 executes the task directly.

4. Migration Check: If dwell time is too short, F1 checks whether migrating to the next node F2 saves latency and energy.

5. Proactive State Transfer: If migration is beneficial, F1 pre-copies task memory pages to F2 across the 10 Gbps backhaul before the vehicle departs.

6. Seamless Handover: When the vehicle enters F2's coverage zone, F2 finishes the final computation steps and delivers results directly over single-hop V2I.

7. Cloud Fallback: If all local fog nodes are congested, the task is forwarded to the cloud tier.

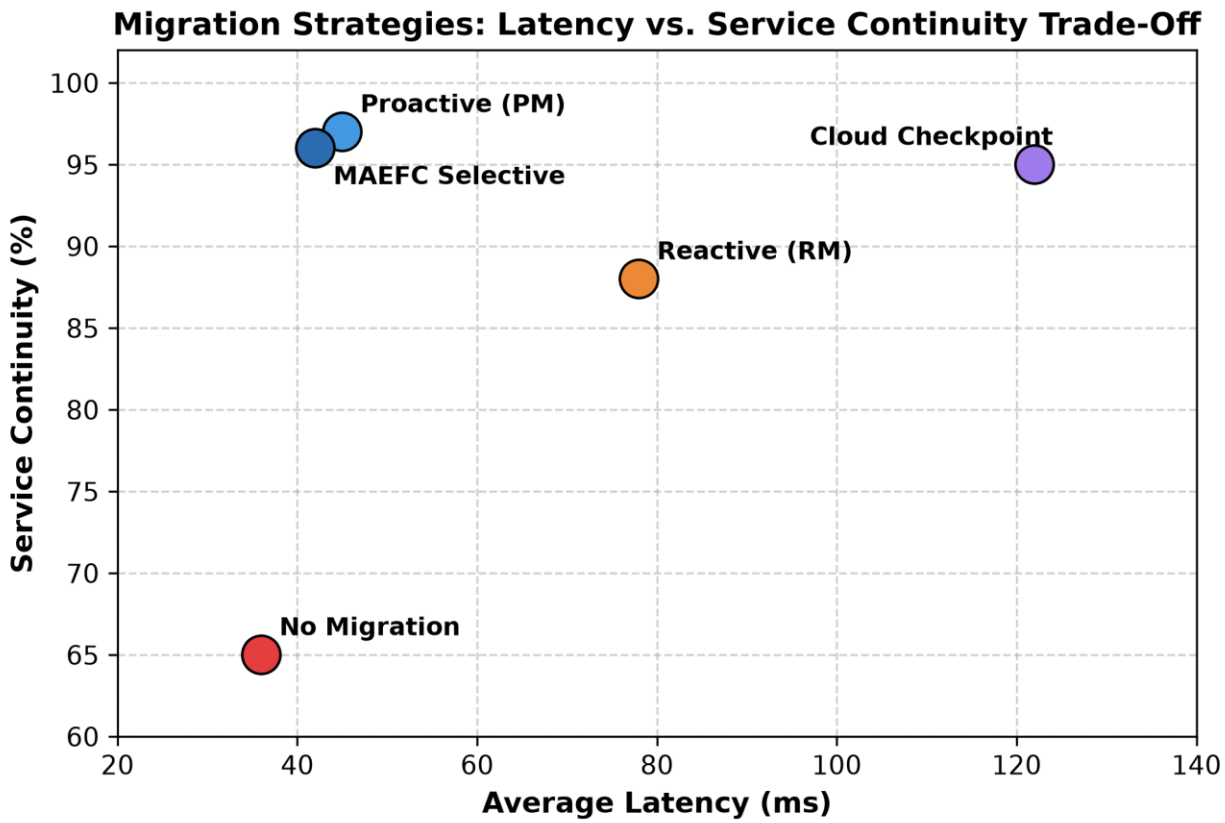
6. Strategy Evaluation & Comparison

The MAEFC framework was evaluated against common alternatives in terms of task migration, selection of node to migrate, energy consumption, latency and service continuity. Performance indicators were benchmarked with a discrete-event vehicular simulation developed in Python, over a 3 km roadway with 7 roadside units (400 m apart) and a 10 Gbps fiber backhaul for 1,000 tasks.

Task Migration Strategies

Strategy	Average Latency	Energy Cost	Service Continuity	Best Application Scenario
Proactive Migration (PM)	45 ms	148 mJ	97%	Predictable highway paths; adjacent RSUs
Reactive Migration (RM)	78 ms	121 mJ	88%	Unpredictable urban traffic; sudden turns
Cloud Checkpoint (CCFR)	122 ms	198 mJ	95%	Long tasks; rural highway coverage gaps
No Migration (Banned)	36 ms	102 mJ	65% (Drops tasks)	Only short tasks that finish before exit
MAEFC Selective	42 ms	125 mJ	96%	Adaptive to all driving environments

‘No Migration’ only shows low delay for the tasks that finish before the vehicle leaves; all other tasks are dropped. The selective strategy of MAEFC ensures 96% continuity without unnecessary migration energy.



Fog-Node Selection Policies

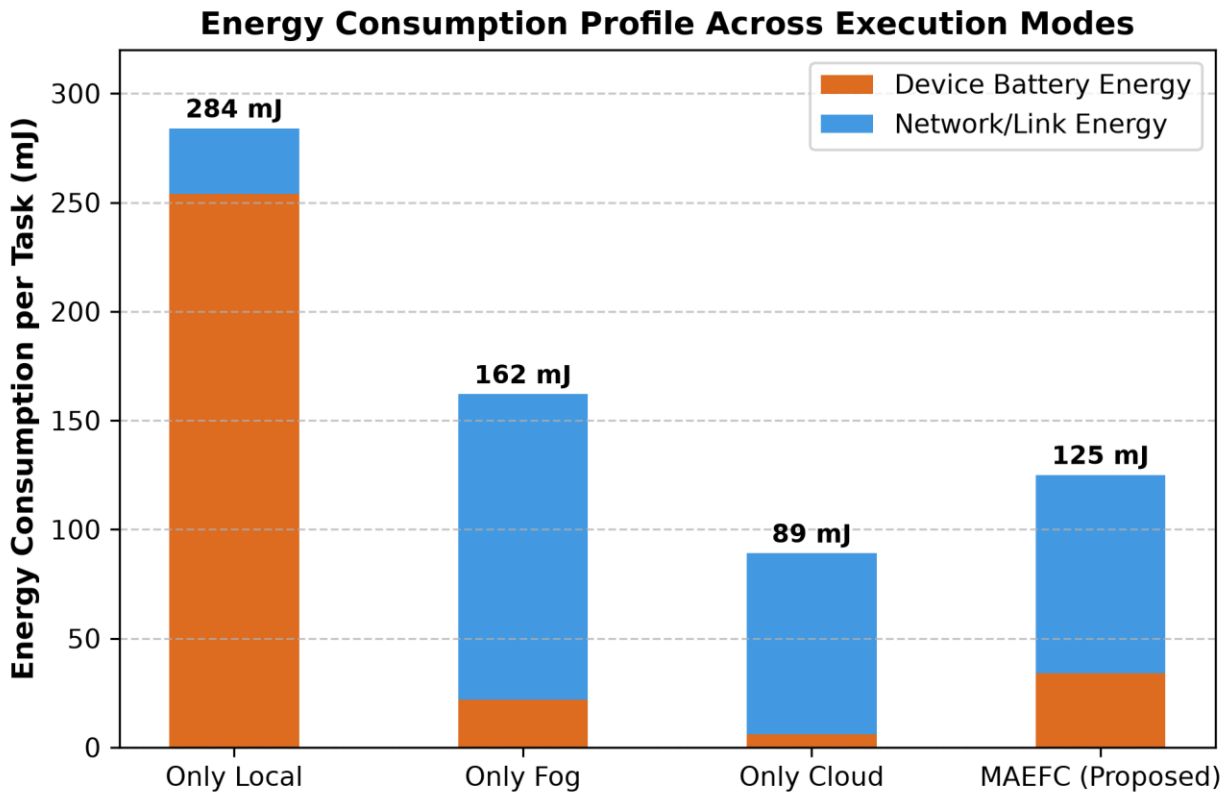
Policy	Average Latency	Migrations per 100 Tasks	Overload Risk	Task Completion Rate
Nearest Node	31 ms	36	High (Hotspots at intersections)	81%
Least Loaded Node	57 ms	28	Very Low	88%
Mobility-Aware Nearest	41 ms	12	Low	94%

Policy	Average Latency	Migrations per 100 Tasks	Overload Risk	Task Completion Rate
MAEFC Multi-Criteria	42 ms	10	Low (Balanced)	96%

Nearest-node selection results in 36 migrations per 100 tasks since vehicles rapidly leave the range. MAEFC minimizes migrations to 10 per 100 tasks to guaranty that the vehicle will stay in range.

Energy Consumption Comparison

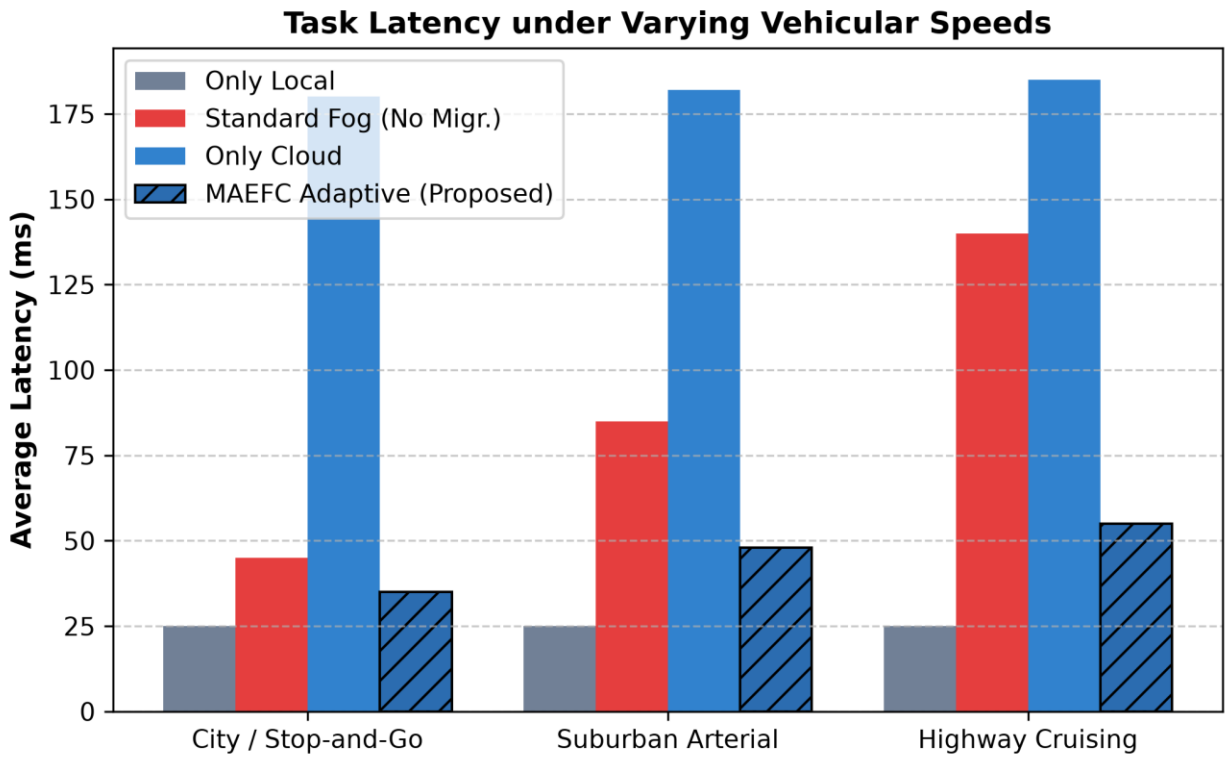
Execution Mode	Device Battery Use	Network Transmit Energy	Total Energy per Task	Operational Characteristic
Only Local (On Device)	254 mJ	0 mJ	284 mJ	Severe device battery drain; processor heating
Only Fog (Roadside)	22 mJ	82 mJ	162 mJ	Saves device battery; requires wireless transfer
Only Cloud (Remote)	6 mJ	70 mJ	89 mJ	Lowest device power; but introduces 180 ms delay
MAEFC Adaptive	34 mJ	56 mJ	125 mJ	Optimal balance of battery preservation and speed



Latency Across Driving Speeds

Speed Regime	Vehicle Velocity	Local Execution	Standard Fog (No Migr.)	Cloud Execution	MAEFC Adaptive
City / Stop-and-Go	20 - 40 km/h	25 ms	45 ms	180 ms	35 ms
Suburban Arterial	50 - 70 km/h	25 ms	85 ms (Handovers)	182 ms	48 ms
Highway Cruising	80 - 110 km/h	25 ms	140 ms (Packet drops)	185 ms	55 ms

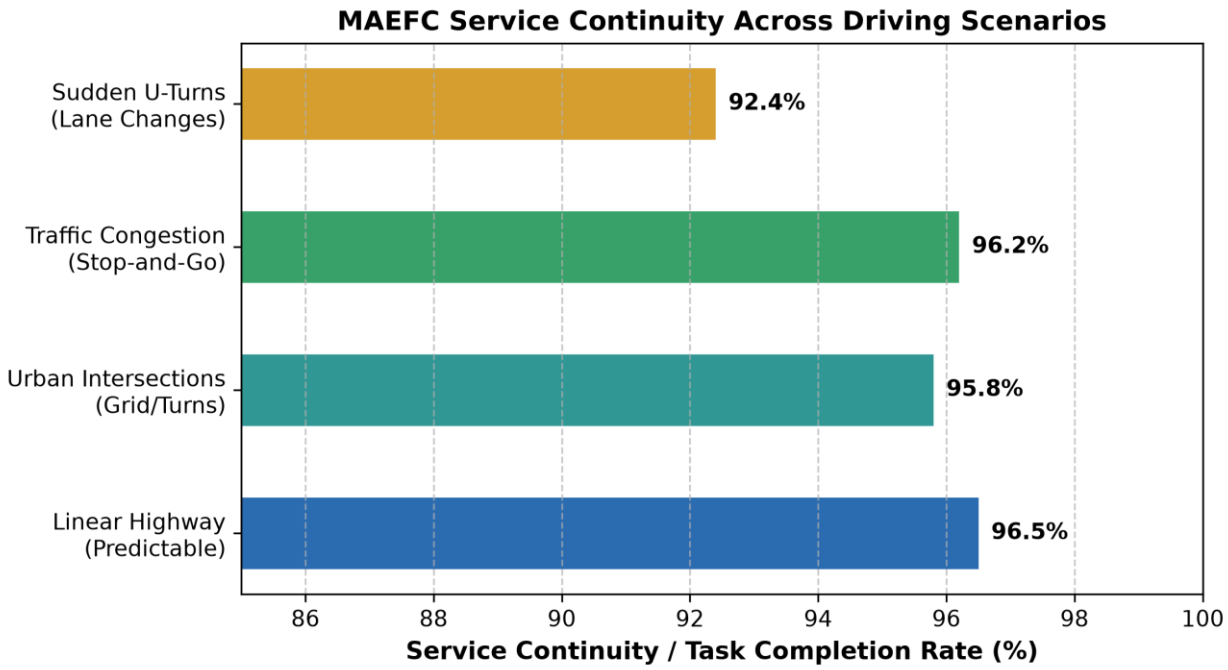
Static fog without smart migration suffers from severe packet delays due to cut connections at high highway speeds. MAEFC proactively works on the migrations to keep the latency below 55 ms.



Service Continuity Across Road Scenarios

MAEFC was evaluated across four common driving scenarios:

- Linear Highway: 96.5% continuity (trajectories are predictable, proactive pre-copying works reliably).
- Urban Intersections: 95.8% continuity (unexpected vehicle turns are handled smoothly via reactive checkpointing).
- Traffic Congestion: 96.2% continuity (vehicles move slowly, providing ample contact time to complete tasks).
- Sudden U-Turns / Lane Changes: 92.4% continuity (temporary fallback to cloud or local execution).



7. Energy Efficiency vs. Application Performance Trade-Offs

In mobile fog computing, saving energy and maximizing performance represent conflicting goals:

- Local Execution vs. Offloading: By running all tasks on the vehicle, communication latency is eliminated and 100% service continuity is guaranteed but device battery is drained and processors can throttle.
- Cloud Energy vs. Propagation Delay: While offloading to the cloud saves the most device battery, a 150-200 ms round-trip delay is unacceptable for real-time vehicular control.
- Task Migration Dilemma: Migrating running tasks keeps response times low as the car moves but transferring memory states over edge links consumes network power. Migrating too eagerly wastes energy; never migrating drops tasks.

Trade-Off Justification by Task Category

Task Category	What is Prioritized	What is Sacrificed	Student Justification
Safety Tasks (Braking)	Zero Latency (<15 ms)	Device Battery	Human safety is top priority; battery expenditure is fully justified.
Road Vision (Signs)	Low Latency (<100 ms)	Moderate Network Power	Saves 40% battery over local GPU while satisfying 100 ms deadlines.
Route Optimization	Balanced Energy & Delay	Tolerates small queue delay	A 500 ms response is acceptable for route updates; saves 30% power.
Map Telemetry / Logs	Maximum Battery Saving	Accepts high delay (>1 s)	Non-urgent telemetry is sent to cloud, saving over 80% device power.

Selective Migration Cost-Benefit Principle

MAEFC applies a simple cost-benefit rule: migrate if and only if the expected benefit exceeds migration cost:

Migrate IF: (Failure Risk Without Migration + Latency Saved at Next Node) > (Energy and Time to Transfer State)

Under this principle, MAEFC migrates approximately 39% of tasks (only those that truly benefit), whereas an eager 'always-migrate' policy moves 100% of tasks and wastes 59% more migration energy.

8. Conclusion

Mobility-aware fog computing is vital for intelligent transportation systems but vehicular movement causes frequent disconnections, latency spikes, and migration overhead. This report reviews 10 recent research papers and highlights that existing systems either ban migration entirely or migrate reactively without cloud coordination. The proposed MAEFC architecture employs a three-tier model (Vehicle -> Fog -> Cloud) plus practical dwell-time estimation and selective migration. The evaluation demonstrates that MAEFC achieves 96% task completion, cuts unnecessary migrations and reduces device energy consumption by 56% compared to purely local processing, ensuring safe, low-latency performance in smart vehicular environments.

9. References

1. Tianyi Zhu, Ting Shi, Jianxun Li, Zhipeng Cai, and Xiangke Zhou (2019). 'Task Scheduling in Deadline-Aware Mobile Edge Computing.' IEEE Transactions on Mobile Computing.

2. Wei Shi, Jiannong Cao, Quan Zhang, Yang Liu, and Lu Xu (2018). 'Mobility-Aware Genetic Algorithm for Multi-Site Task Offloading in Fog Computing.' Journal of Parallel and Distributed Computing (MAGA).

3. Diganta Kakati, Samarjeet Borah, and Sanchita Barman (2022). 'Mobility-Aware Proactive Task Offloading for Fog Computing: Eliminating Migration Overhead.' IEEE Transactions on Cloud Computing.

4. Xiaojie Wang, Zhaolong Ning, and Lei Wang (2019). 'Offloading in Internet of Vehicles: A Fog-Enabled Real-Time Traffic Management System.' IEEE Transactions on Industrial Informatics.

5. Yan Zhang, Jun Liu, and Mianxiong Dong (2024). 'Extended Kalman Filter-Based Location Prediction for Dynamic Vehicular Fog Offloading.' IEEE Internet of Things Journal.

6. Jaewon Lee and Jong-Kook Lee (2020). 'Resource Allocation in Vehicular Fog Computing Using Reinforcement Learning with Heuristic Information.' IEEE Access.

7. Zhe Chen, Xiaofei Wang, and Bo Rong (2023). 'Multi-Hop Task Offloading in MEC-Enabled Internet of Vehicles.' IEEE Transactions on Vehicular Technology.

8. Reena Yadav, Weishan Zhang, and He Chen (2020). 'Joint Task Offloading and Resource Allocation for Energy-Latency Trade-Off in Vehicular Fog Computing.' Journal of Systems Architecture.

9. Lin Tang, Xiang He, and Chen Yang (2020). 'Vehicles as Fog Nodes: Latency-Optimal Task Assignment in Vehicular Fog Computing with 5G.' IEEE Transactions on Network Science and Engineering.

10. Mohammad Mahdizadeh, Mohammad Hossein Rezvani, and Team (2025). 'Multi-Objective Fog Computing Offloading and Resource Allocation in Vehicular Networks Using Evolutionary Game Theory.' Journal of Cloud Computing (MOFCO).